

JavaFX 2

Indice

Sommario

Indice.....	1
Introduzione.....	2
Interfaccia grafica secondo Java FX.....	2
Nodi Parent e i nodi Controller.....	2
La classe Node.....	3
Sviluppo di un'interfaccia grafica.....	4
Attraverso il codice Java.....	4
Cosa accade:.....	4
Attraverso un file FXML.....	4
Struttura di un'applicazione JavaFX 2.....	6
Nuovo progetto JavaFX2.....	6
SceneBuilder.....	7
Il supporto di java.....	8
Eventi e controller.....	9
I controller in FXML.....	10
Cosa accade nel file FXML.....	11

Introduzione

L'API JavaFX 2 é una serie di librerie in cui esiste la possibilità di disegnare un'interfaccia grafica attraverso un file di testo fxml molto simile all'html.

Non solo, è possibile modificare lo stile di tutti gli oggetti nella nostra interfaccia modificando un foglio di stile CSS (proprio come accade per le pagine html).

L'obiettivo di Oracle è quello di sostituire il pacchetto SWING con il nuovo pacchetto JAVAFX 2 perché lo SWING richiedeva molte linee di codice e spesso poco intuitive per fare una banale interfaccia grafica.

Per utilizzare le librerie di JavaFX 2 è sufficiente installare la JDK dalla versione 7 in su.

Interfaccia grafica secondo Java FX

Un'interfaccia secondo JavaFX2 è composta da tre elementi:

- Stage : che si riferisce alla finestra del sistema operativo (es. di windows)
- Scene : E' il contenitore principale dove andremo a comporre la nostra grafica, che è fatta da una serie di contenitore e di controlli.
- Node : Può essere o un contenitore o un controllo da visualizzare sulla scena. I nodi devono essere costruiti in modo gerarchico (un nodo root, n nodi padre, n nodi figli, ecc... fino ad arrivare ai nodi foglia che sono i nostri controlli come ad es. un button, una casella di testo, ecc...).

Nodi Parent e i nodi Controller

Esistono due tipi di nodi, il nodo contenitore ed il nodo controllo:

Il nodo contenitore si riferisce all'impaginazione dei controlli all'interno di un altro nodo contenitore fino ad arrivare al nodo root che corrisponde alla scena (il famoso getContentPane dello SWING).

I nodi controllo si riferiscono a tutti gli oggetti di controllo (button, casella di testo, etichetta, ecc...) contenuti in un singolo nodo contenitore.

Tabella tipi di nodi contenitori

Some 'Parent' Nodes (Container 'Panes')

- ▶ **BorderPane** (5-areas)
- ▶ **Hbox, VBox** (linear sequence)
- ▶ **StackPane** (overlay all children)
- ▶ **GridPane** (row x columns)
- ▶ **FlowPane** (flowing boxes, wrap around)
- ▶ **TilePane** (flowpane with equally sized boxes)
- ▶ **AnchorPane** (magnetically attach nodes at corners or sides)



Il compito di questi contenitori è quello di organizzare la grafica in modo da mettere ogni controllo in una specifica posizione.

Java Fx I controlli più usati



Per completezza sull'argomento potete dare un'occhiata alla documentazione ufficiale di oracle sulle librerie JavaFX2 e su tutti i tipi di controlli possibili:

- <https://docs.oracle.com/javafx/2/api/>
- http://docs.oracle.com/javafx/2/ui_controls/jfxpub-ui_controls.htm

La classe Node

E' una classe astratta che può essere vista come una pagina con uno specifico scopo, esempio quello di visualizzare un immagine o quello di visualizzare dei controlli, ecc...

Documentazione oracle sulla classe node

javafx.scene

Class Node

java.lang.Object

javafx.scene.Node

All Implemented Interfaces:

EventTarget

Direct Known Subclasses:

Canvas, ImageView, MediaView, Parent, Shape

- MediaView: una pagina che visualizza un video
- ImageView: una pagina che visualizza un'immagine
- Canvas: una pagina che visualizza un gioco
- Shape: una pagina che visualizza un grafico (es. un grafico a torta)
- Parent: una pagina che contiene un'altra pagina o controlli.

Sviluppo di un'interfaccia grafica

Con la classe Java FX 2 ci sono due modi per sviluppare un'interfaccia grafica :

- il primo è quello di utilizzare il codice Java
- Il secondo è quello di utilizzare come sorgente di una scena un file FXML passato come parametro alla scena.

Attraverso il codice Java

Un mio applicativo sviluppato con interfaccia grafica JAVAFX 2 deve avere :

1. una classe Main, sottoclasse della classe astratta Application, che ci obbliga a definire un metodo start.
2. Il metodo main statico che ha un metodo launch per avviare la finestra grafica principale.

Il metodo launch

```
public class Main extends Application {  
  
    @Override  
    public void start(Stage stage) {  
        Group root = new Group(); // the root is Group or Pane  
        Scene scene = new Scene(root, 500, 500, Color.BLACK);  
        stage.setTitle("JavaFX Demo");  
        stage.setScene(scene);  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Cosa accade:

1. Il metodo launch crea automaticamente una finestra (stage) non visibile all'utente, già bella e pronta;
2. chiama il metodo start con argomento la finestra appena creata, in modo da avere un riferimento ad essa.
3. Creare la nostra pagina principale da visualizzazione, la scena.
4. Settare la scena della nostra finestra con quella appena creata.
5. Rendere visibile la finestra(show()) agli utenti.

Da questo momento in poi il nostro programma non è altro che una serie di eventi che consentono l'interazione tra l'utente e il nostro applicativo attraverso i controlli.

Attraverso un file FXML

Un file di testo FXML ci consente di sviluppare un ambiente grafico nello stesso modo in cui si sviluppa una banale pagina web.

Esempio di file FXML

```
<HBox id="HBox" alignment="CENTER" spacing="15.0"
AnchorPane.rightAnchor="23.0" AnchorPane.topAnchor="22.0">
  <children>
    <Button id="button1" fx:id="newIssue" onAction="#newIssueFired"
      text="New" />
    <Button id="button2" fx:id="saveIssue" onAction="#saveIssueFired"
      text="Save" />
    <Button id="button3" fx:id="deleteIssue" onAction="#deleteIssueFired"
      text="Delete" />
  </children>
</HBox>
<ImageView id="IssueTrackingLite" layoutX="14.0" layoutY="20.0">
  <image>
    <Image url="@IssueTrackingLite.png" preserveRatio="true" smooth="true" />
  </image>
</ImageView>
```

1. Creo una pagina a sequenza di box orizzontali (HBOX)
2. Al suo interno ci metto 3 bottoni
3. Affianco all'HBOX c'è un'immagine

Con il programma SceneBuilder possiamo creare la nostra interfaccia grafica FXML in modo visuale senza programmare nulla e dopodiché sarà sufficiente dire che il sorgente della nostra interfaccia grafica è il file FXML che si genera in automatico proprio in sceneBuilder.

Caricamento del file FXML

```
public void start(Stage stage) throws Exception {  
    Parent root = FXMLLoader.load(  
        getClass().getResource("circle.fxml"));  
  
    stage.setTitle("Circle Demo");  
    stage.setScene(new Scene(root, 500, 150));  
    stage.show();  
}
```

La classe FXMLLoader è in grado di caricare il contenuto di un file FXML nel nostro programma e di convertirlo in un oggetto di tipo parent (contenitore).

La scena non è altro che il contenitore principale di una finestra, possiamo quindi settare la scena ad una nuova scena che ha come contenitore quello generato dal mio file FXML.

Struttura di un'applicazione JavaFX 2

Vediamo cosa riusciamo a fare in poco tempo con il supporto delle librerie JavaFX2 e con questo strumento così potente e semplice da utilizzare, scenebuilder.

Innanzitutto inquadreremo le varie fasi di progettazione di un ambiente grafico con Java Fx2:

- o Progettazione del nostro scenario utilizzando sceneBuilder
- o Sviluppo del codice Java
- o Applicazione di uno stile al nostro scenario mediante un file CSS

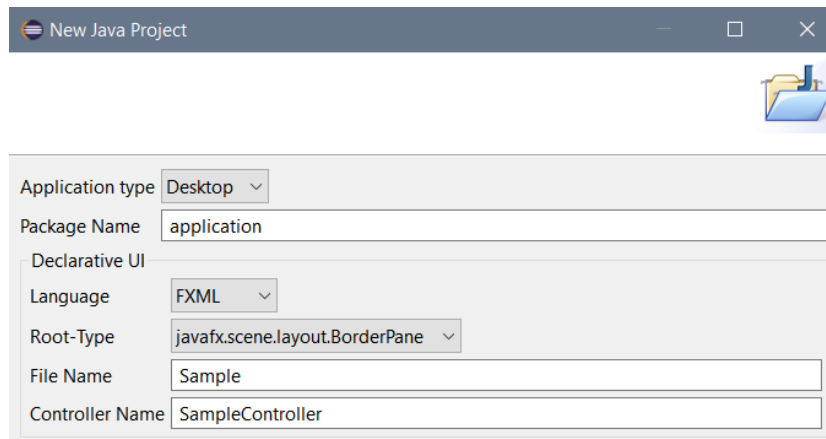
Nuovo progetto JavaFX2

Una volta installato per bene, sia l'applicativo sceneBuilder, sia il software aggiuntivo per eclipse sarà piuttosto semplice iniziare un nuovo progetto JavaFX2.

Tra le voci di menu in nuovo progetto in eclipse troverete una voce nuova JAVA FX; all'interno c'è la possibilità di selezionare un progetto di tipo JavaFX ed è proprio quello che ci occorre.

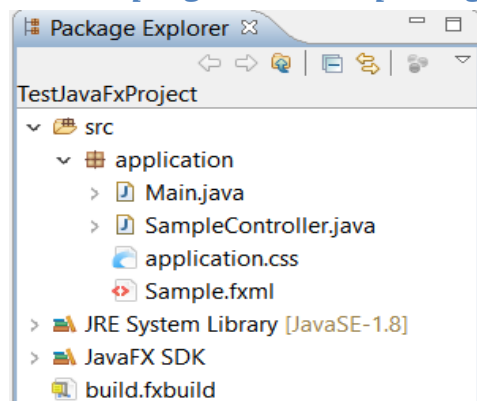
Su un nuovo progetto JavaFX, facendo Next ci troveremo nella seguente pagina:

nuovo progetto JavaFX



- Application type: possibilità di definire se la nostra interfaccia grafica sarà sviluppata per un Pc o per un cellulare.
- Name Package: è il nome del pacchetto che conterrà le due classi java (Main, Controller) del progetto JavaFX, il file FXML ed eventualmente il file CSS.
- Language : il linguaggio usato è l'FXML
- Root-type: Il tipo di contenitore principale, la scena, (VBOX, HBOX, BORDERPANE, ecc...)
- File name: Il nome del file FXML
- Controller Name: Il nome della classe controller (la classe che contiene la logica di funzionamento dei controlli nell'interfaccia, la gestione degli eventi).

Crea un progetto con un package al cui interno ci sono 4 file.

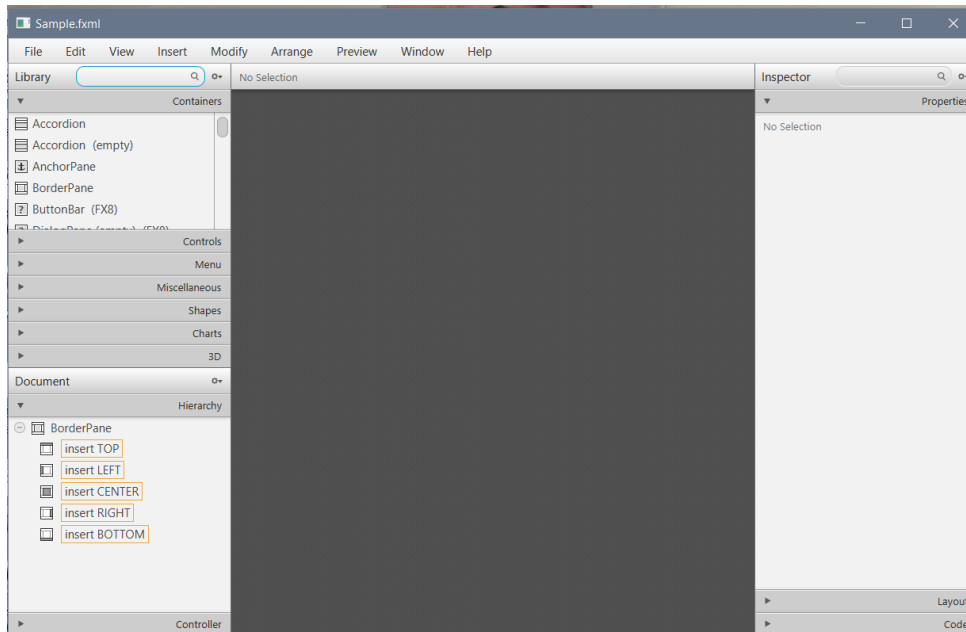


Il file su cui dobbiamo concentrare la nostra attenzione per il momento è il file FXML che è l'artefice principale dell'interfaccia grafica del nostro progetto.

Inizialmente si presenta con un codice di base che rappresenta una finestra vuota con un singolo scenario principale di tipo BorderPane perché è quello che abbiamo scelto nella finestra di progetto JavaFX precedente.

SceneBuilder

Come si presenta SceneBuilder



Non starò a spiegarvi sceneBuilder perché è talmente intuitivo che preferisco concentrarmi sullo sviluppo.

Il file sample.FXML che aveva una finestra Vuota di tipo borderPane, dopo le modifiche con scenebuilder avrà del codice aggiuntivo che rappresenterà tutti i nodi parent e i controlli che noi abbiamo aggiunto successivamente al borderpane di partenza.

Ovviamente tutti i nodi sono accessibili tramite codice Java ed ogni nodo ha in dotazione tutta una serie di attributi e metodi suoi o ereditati.

Per arrivare ad un controllo specifico è necessario navigare tutti i nodi padri, partendo dal nodo root oppure attraverso un id identificativo che ci consente di riferirci direttamente al nodo interessato.

Il supporto di java

Java fornisce sempre un'ampia documentazione di tutte le classi, in particolare vi scrivo un link dove troverete sia documentazione che esempi sul JavaFX.

<http://download.oracle.com/otndocs/products/javafx/2/samples/Ensemble/index.html>

In particolare proprio in questa guida potete notare che ad esempio la classe Button eredita ed effetto domino tutte le proprietà ed i metodi delle superclassi che la precedono:

Esempio di classe Button



Noterete inoltre che tutto deriva dalla classe `Node`, ne consegue che un node a compiletime, a runtime potrà essere qualsiasi elemento grafico.

Eventi e controller

L'evento è una classe java `Event`. Un evento per generarsi necessita di due cose:

- Chi lo genera. Ovvero il nodo che genera l'evento
- Cosa è stato fatto. Ovvero che tipo di azione è stata fatta sul nodo

Tutti i nodi padre dello stesso nodo sono tutti coinvolti nella generazione dell'evento.

Una volta creata l'interfaccia bisogna focalizzare l'attenzione:

- Su quali nodi generano eventi
- Che tipo di eventi sono stati generati.

Ad esempio su un nodo `Button` voglio fare delle cose se l'utente preme su di esso, generando un evento di tipo azione (on Action).

In cascata dal nodo foglia, il controller, andando verso il root, ad ogni nodo viene chiesto se ha un gestore evento attivo; ne consegue che un evento sul root si ripercuote su tutti i nodi figli e se vogliamo un gestore evento su un gruppo di nodi sarà sufficiente gestire l'evento sul nodo padre. Posso tuttavia propagare l'evento anche da un nodo figlio ad un nodo padre.

Se vogliamo catturare un evento, dobbiamo creare una classe che implementa l'interfaccia `EventHandler` la quale dichiara i metodi per gestire un evento.

Solo l'evento registrato sarà gestito come evento reale:

setOnEvent-type (EventHandler<? Super event-class> value> → una qualunque sottoclasse di eventi alla quale appartiene.

Esempio di evento associato al Button

```
Class ButtonActionHandler implements EventHandler<ActionEvent> {  
    public ButtonActionHandler(/*param*/) {  
    } /* costruttore della classe non necessario */  
    @override  
    Public void handle(ActionEvent event) {  
        Button b = (Button) event.getSource();  
        String buttonText = b.getText();  
    }  
}
```

Esempio precedente ma con classe anonima

```
Btn.setAction(new EventHandler<ActionEvent>() {  
    Public void handle(ActionEvent event) {  
        Button b = (Button) event.getSource();  
        String buttonText = b.getText();  
    }  
});
```

Cosa è successo? Anziché creare una classe ad hoc che implementa un'interfaccia EventHandler; creo direttamente l'evento di tipo Action con l'unico metodo che conterrà il codice implementativo di ciò che deve essere fatto quando l'evento accade.

La classe anonima è una classe interna senza nome che si auto-istanzia creando un unico oggetto, corrispondente al new EventHandler.

Questa classe diventa molto utile quando le linee di codice sono poche per questo è molto frequente vedere codice di gestione eventi fatto in questo modo.

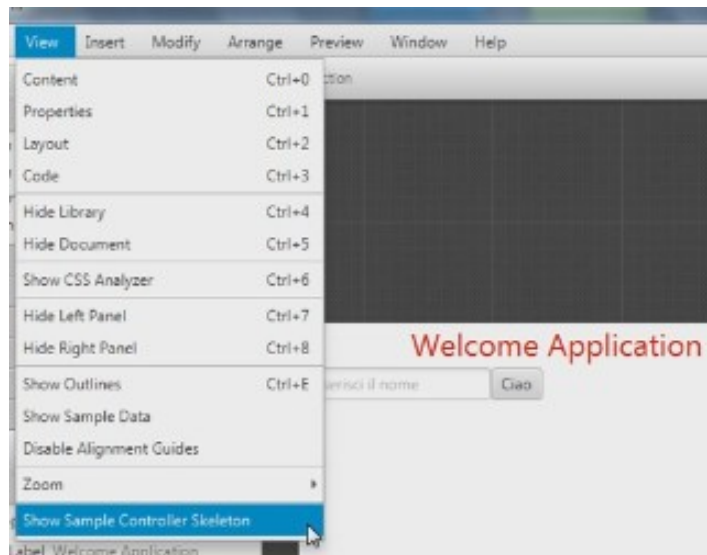
I controller in FXML

Il controller è una classe java associata ad una scena. Questa classe contiene sia i riferimenti a tutti nodi di una scena e sia la registrazione degli eventi gestiti.

Il nodo a cui vogliamo associare un evento può avere un riferimento `fx:id` che ci consente di accedere direttamente al nodo che genera l'evento.

Nell'esempio che vi mostrerò, vi faccio vedere un preview di quello che sarà la classe controller, direttamente nel programma sceneBuilder che tra le sue funzioni ha quella che ci permette di vedere l'anteprima del codice necessario nella classe controller per avere un riferimento ad un controllo o il codice necessario per gestire un evento.

Funzione di sceneBuilder



Esempio di codice della classe controller

```
Sample Skeleton for 'Sample.fxml' Controller Class

package application;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.TextField;

public class SampleController {

    @FXML
    private TextField txt;

    @FXML
    private Button btn;

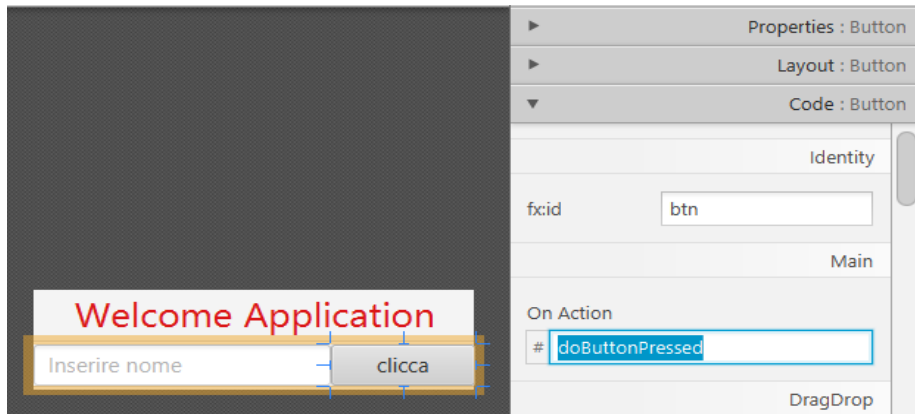
}
```

Una volta identificato il nodo che genera un evento, assegnandogli un `fxid`, in automatico crea un campo privato nella classe di tipo controller che ci consente di puntare direttamente al nodo corrispondente.

Adesso non dobbiamo fare altro che scrivere un metodo che contiene il codice da eseguire quando l'evento si verifica.

SceneBuilder ancora una volta ci viene in aiuto perché nel suo menu a tendina ci elenca tutti gli eventi che genera un nodo, basta scegliere quale evento vogliamo gestire e decidere il nome del metodo che lo gestisce.

Programmare un evento quando premo sul bottone



doButtonPressed è il metodo che contiene il codice da eseguire quando si preme il pulsante “clicca”.

Il codice presente nel preview viene modificato, il metodo che abbiamo scelto è stato auto-generato:

```
@FXML
void doButtonPressed(ActionEvent event) {
}
```

Ora, dato che la classe controller dovrà essere esattamente uguale al nostro preview di sceneBuilder, basta copiarlo in eclipse nella classe controller ed il gioco è fatto.

Cosa accade nel file FXML

Crea un riferimento alla classe controller

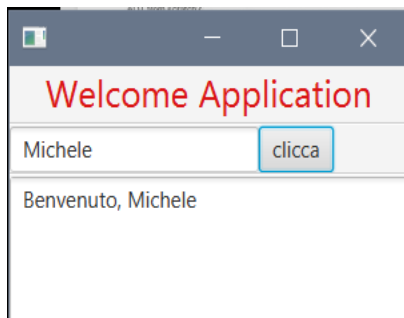
```
<BorderPane                                prefHeight="69.0"                                prefWidth="299.0"
xmlns="http://javafx.com/javafx/8.0.60"    xmlns:fx="http://javafx.com/fxml/1"
fx:controller="application.SampleController">
```

Crea un riferimento per i controlli TextField e Button

```
<TextField fx:id="txt" prefWidth="350.0" promptText="Inserire nome" />
<Button    fx:id="btn"    mnemonicParsing="false"    onAction="#doButtonPressed"
prefWidth="246.0" text="clicca" />
```

Il campo privato nella classe controller è proprio txt e btn; non solo, viene anche detto che l'evento onAction generato sarà gestito nel metodo #doButtonPressed.

Esempio di codice in doButtonPressed



```
@FXML
void doButtonPressed(ActionEvent event) {
    if (txt.getText().length() != 0)
        area.appendText("Benvenuto, "
            +txt.getText()+"\n");

    else
        area.appendText("Benvenuto, \n");
}
```